Idea to reality 2026

Assignment #3

General instructions:

- Make sure to read all the exercises instructions and to follow the instructions step by step by the order that I wrote.
- Make sure to all open the links that are in the assignment and read them.
- Do not wait to work on this assignment, start working on it whenever you can as it might take more time than you think.
- I work on these assignments a lot just for you guys and this is probably the most practical course you will have in the university, so I highly recommend that you really devote your time and effort and do your best to do them properly. It will benefit you a lot.
- If for some reason you weren't in class, please go over the slides and videos before starting this assignment.
- Solve the exercises by yourself. You can consult with friends when needed but **don't share your code with your friends**. Of course copying from others is unethical and will be reported.
- Solve the exercises using your self defined 'baby steps'. Don't start with a program that does everything. You can always create another small project in vs code just to make sure something works and then get back to your 'regular' project.

For each coding exercise, before you start the exercise, please create a **private** github repository just for that exercise and invite me to that repository (my github user is dk8827). While you are working on the exercise, once in a while(let's say every 30 minutes or so) or after you make any non-trivial change in your code, you should **commit and push to github your changes**. This will help you a lot in understanding how you work, what you did wrong etc.
Every time before you commit code, go over it(check all the files that changed and the diff of the content itself) and make sure that the commit makes sense.
Do not simply commit code that came out from an LLM. Sometimes LLMs do stupid things like removing your comments, deleting parts of your code, or adding some things that don't really make sense…

If you never worked with git and vscode, please watch/do an online tutorial like this

If it's a group exercise- make sure that all group members commit and push to that git repository, each one using his own github user. Not committing and pushing to git properly will reduce your grade. Also, your commit messages must be meaningful. Here are some best practices for git commit messages:
https://dev.to/sheraz4194/good-commit-vs-bad-commit-best-practices-for-git-1plc

Exercise 1 - a real-time scraper of shopping websites

In this exercise you will implement a real-time scraper of shopping websites.

You will build a minimal website using Next.js. The site will use a backend server that you should write using FastAPI. The scraping logic should be written in Python inside the FastAPI server.

In this exercise I will guide you less than in other exercises, because I want you to build your self-confidence. I will give you the minimal requirements for this project. Feel free to expand it with more things if you wish.

The website should let the user type a product name in a text box and click a 'Find product' button.

Your program should go over multiple e-commerce websites, extract product information from them, and show the results to the user.

The websites are:

Amazon.com
BestBuy.com
Walmart.com
Newegg.com

Your scraper should work well on at least 3 out of these 4 websites.

For example, if the user searched for:

Lenovo Tab P12-2024

and your scraper is currently scraping Amazon, then your scraper should open the Amazon search page:

https://www.amazon.com/s?k=Lenovo+Tab+P12-2024

It should take a relevant result from the search page, open that product page, and scrape the information from it.

The information that you should display to the user is a simple table:

Website	Product title	Price	Average rating	Review count	Status	Method
Amazon.com	Lenovo Tab P12-2024 - Expansive Touchscreen Tablet...	332.54	4.7	315	Success	Browser
BestBuy.com	...	...	...	...	Success	Basic
Walmart.com	...	...	...	...	Success	Firecrawl
Newegg.com	...	...	...	...	Failed	N/A

A very important real-world issue: different websites may return different products for the same query so think how you want to make sure that you get the most similar product every time.

If a website fails, your whole system should not crash. Show that this website failed and continue showing the results from the other websites.

Scraping methods and fallbacks

Your scraper must implement all four of the following scraping/extraction methods:

Basic scraping
Use requests + BeautifulSoup, Scrapy, or another direct HTML parsing method.
Browser-based scraping
Use Playwright or Selenium to load the page like a real browser and extract the HTML/content by parsing it.
LLM-based extraction
Give the relevant HTML or visible page text to an LLM and ask it to extract the required fields into structured data.
Firecrawl
Use the Firecrawl API, or another similar scraping/extraction API, as an external scraping service.

Your backend should automatically use these methods as a fallback pipeline.

For each website, your scraper should try them in order:

Basic scraping
↓ if failed or important fields are missing

Browser-based scraping
↓ if failed or important fields are missing

LLM-based extraction
↓ if failed or important fields are missing

Firecrawl / external scraping API
↓ if failed

Return a clean failure for that website

Your system should try the next method if:

The page is blocked
The page did not load correctly
The HTML is missing important data
The product title, price, rating, or review count could not be found
The extracted values do not make sense

The frontend should show which method finally succeeded for each website.

If all four methods fail for a website, your system should not crash. It should show that this website failed and continue showing results from the other websites.

Important note about scraping

In general, when scraping, I highly recommend that you open the HTML code and go over it with your own eyes, with the help of LLMs, to see what is going on there.

Extra feature

Of course, this website is very simple, so feel free to add to it whatever you want, feature-wise, logic-wise, or styling-wise.

Add at least one extra feature or improvement. Whatever you decide.

I highly recommend that you first implement the logic in Python and only after you see that everything works, implement the basic website that calls the FastAPI backend and displays the information.

Exercise 2 - Real-time multiplayer trivia competition
(Reminder: Remember the 'wow' factor)

In this exercise I will again give you initial guidance and guidelines but you can implement it as you wish. This will be your first "baby"

You need to implement a real-time multiplayer trivia competition.

First you should use a "smart" LLM (like gpt 5.5) to generate for you a 'database' of general trivia questions and answers. Create at least 250 questions(and a difficulty rating between 1-10 to each question) and answers and 3 more 'wrong' answers for each question using that LLM. Then have a smart LLM go over those questions and make sure all questions are unique (no repetitions). Then have less smart LLMs (for example gemma in multiple sizes or any LLMs that you choose) try to solve each of these questions. If the 'less smart' LLMs were not able to answer correctly, remove that question from the 'database' as it might be too hard.

Run the above using any tools that you want once(of course do it with code and not manually).

Remember that when you ask an LLM for 'many' things in a single prompt the quality of the answer might become weak so you might need to batch it to multiple requests.

(Sometimes when LLMs create multiple possible answers to questions they make the correct answer longer than the others so make sure to fix it if that's the case).

So the output of everything above should be a csv of at least 250 questions with a correct answer to each one and 3 extra 'wrong answers' and a difficulty column.
Then you should take this data and convert it into a simple sqlite database.
(Remember- commit and push to your private github)

You should implement the multiplayer server(backend) using python and **socket.io**.
However, you can implement the frontend with **any** technology that you wish (for example- next.js, react etc) and using any tool.

The competition should work like this:
The clients connect to the server and the server waits for 30 seconds for the matchmaking of the user against other users. After the time ends, the game begins with all the users that waited. A random trivia question is shown for a couple of seconds (you decide) on all of the players' screens and 4 options are displayed. The user should choose the correct answer. The users that answer faster should get more points. If the user answered incorrectly, he should get 0 points. Each game will include 10 questions and after those questions there will be shown a leaderboard with all the scores and winner. If just a single human player is in the game, you should add between 1-3 bot users to the game so he won't play against himself.

Of course make sure that the bots don't answer too quickly and that they sometimes have mistakes.
In each game, each player will get 3 different 'helps' (one of each) : one is 50/50 that removes 2 of the incorrect answers. The second is 'call a friend' (you should use any LLM via api (use PydanticAI for the call) and ask him to give advice for the player, make it fun). The third is 'Double the score' which will double the score of that question (players should use it strategically when they are certain they know the answer).
During the game, the players will be able to send messages to each other(basic chat) and/or send emojis.

In order to make the game more interesting, make the difficulty adaptive to the human players. If the human players answer a question correctly then the next question should be harder (and if incorrectly, the next question should be easier).

You should have a sqlite database that will save some data(you decide which data and for which purposes).

When everything runs fine, you can use https://pinggy.io/ (or ngrok) to get a url for your friends to connect to your pc and play with you the game. You can also of course upload everything to a cloud provider like Render/azure etc (completely optional but highly recommended). Play with some friends/family and listen to what they have to say about the competition.

You can add any other things that you wish to the project including any kind of feature, animations, graphics, mobile phone support or really anything that you feel will make it great.

Add at least 2 extra features/changes to what I wrote.

As this exercise is 'your own' make sure to invest some time in making it your own (your own style, changes from feedback that you got from others etc).

Exercise 3 - Training a Neural Network

In this exercise you should create a python program that trains a Neural Network using Keras.

Choose any **interesting** problem that you want your neural network to solve. Try to be original. You can think of your own hobbies, things that will be awesome to see etc. If you do use LLMs for ideas, don't take the first ideas that they offer- guide them into fields that you like, Drill in, give feedback and try to think of something awesome and not 'classic'. We will be having a competition on the most amazing solutions so do something cool.

Example fields:

- Image
- Sound
- Game AI
- Computer vision
- Predictions

You have two options to train your neural network- (choose one, i personally think that the genetic algorithm is much more interesting and amazing to talk about in job interviews)

1. Supervised Learning from a Dataset. Gather input and output examples (dataset). Just for example images of cats and images of dogs with each one already classified as dog or cat (Of course this is a very boring idea). Then train the neural network using backpropagation or similar methods so it can correctly predict an input that was not in the training set.
2. A Genetic Algorithm. Start with randomly weighted neural networks, find a way to run and score each neural network (fitness), choose the best ones- breed, mutate and so on for multiple generations. This method allows you to find solutions to problems when you don't have examples of specific inputs and outputs like in option 1 above (and is super cool).

If you choose option 1 above, you have to find/augment/create good datasets. Try to see if you can find a good dataset to train on. You can find many datasets online(for example in kaggle, statistics websites etc). You can of course write a python that scrapes data from online websites and creates a dataset for your idea. You can of course also have a smart LLM create you a dataset but make sure that the dataset is diverse enough. You can also use things like Moondream to help you create a dataset. You can also read about dataset augmentation to expand your dataset using code (so it will be even more diverse).

If you choose option 2 above, you have to think well about your fitness function that will grade which solution (neural network weights) is best for you.

Implement the training in python and add a visual chart that is updated during the training. The chart should show how your training progresses:

- If it's option 1 above, show the accuracy on the training dataset and on the validation dataset. If you see that the accuracy on the training dataset is high and is not increasing on the validation dataset that is probably over-fitting so try to change things and/or stop the training when you see that the validation accuracy is decreasing.
- If it's option 2, show how the fitness is improving.

When your training is finished and you have the model ready, save the model weights into a file.

Your program should be a real program that gives some real value or simply has some "wow" factor. You can create a gui for your program in any way you wish. One option is to create the gui using Gradio.

(completely optional: if you wish you can convert your keras model to tensorflow-lite and then use things like flutter + tflite_flutter to create a mobile app that can run your trained neural network in realtime on your phone)

Video of this exercise will be used for the competition so make sure you are ok with other students watching it.

Exercise 4 - final ideas V2

Add to this sheet your final(for now) project ideas. You should add between 2-4 ideas. I have added columns that will help you see if the idea is valid. Make sure to go over all the columns and make sure that your product has 'Yes' in all the columns (You need to manually write 'Yes').

Submission instructions:

- Zip the code of exercise1 into a file called assignment3-exercise1.zip
- Zip the code of exercise2 into a file called assignment3-exercise2.zip
- Zip the code of exercise3 into a file called assignment3-exercise3.zip
- Upload these zip files to Moodle. (if you see that your zip is large do not include .venv or other large directories that are not needed for the project to run).
- Make a high quality video screen recording of exercise1 running properly with all its features. Make sure that you go over all the important features. Use any screen recording software for that. Make sure that the video looks ok, understandable and the quality isn't bad. There are many such free recorders for windows and for mac.
- Make also a video of exercise 2,3 running properly with all the features.
- Video of exercise 2 will be used for the competition so make sure you are ok with other students watching it.
- Remember to add to the sheet in ex4.
- Upload each video file to google drive and create links that are open to **anyone with the link**
- **Fill in this google form with the information about your exercises and the links to the videos.**

Of course - filling the form with incorrect information is not allowed(for example, saying that everything runs fine while it doesn't) and will have a major impact on your grade.